37. gRPC and Protobuf Intro

37. gRPC and Protobuf Intro

Phase: Phase 6 — Systems and infrastructure
Estimated time: 2 days
Slug: `37-grpc-protobuf`

Learning objectives

- Explain protobuf IDL and codegen
- Sketch a gRPC service in Go
- Compare gRPC vs JSON HTTP
- Handle deadlines via context

Prerequisites

Complete prior numbered lessons. Type every example yourself.

Why gRPC?

- Strongly typed contracts (`.proto`)
- Efficient binary encoding
- Streaming RPCs
- First-class deadlines/status codes

Flow

1. Write `service.proto`
2. Generate Go with `protoc` / `buf`
3. Implement server interface
4. Dial with client stubs

vs REST/JSON

Use gRPC for internal service meshes and streaming; REST/JSON for public browser-facing APIs often still wins.

Contexts carry deadlines across gRPC calls — wire them end-to-end.

Common mistakes (especially from Python instincts)

- Exposing gRPC directly to browsers without grpc-web/gateway
- Ignoring status codes (use codes.NotFound etc.)
- Huge protobuf messages without pagination

Exercises

Work in `exercises/37-grpc-protobuf/`. Read that folder's `README.md`.

- **A (Drill):** make unit tests pass with the simplest correct code.

- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/37-grpc-protobuf/` until you have an honest failing attempt.

Self-check

1. What does codegen produce?
2. When prefer gRPC?
3. How deadlines propagate?

Further reading

- https://grpc.io/docs/languages/go/quickstart/